`std::thread` instance that has been used as the source of a move is equivalent to a default-constructed `std::thread` instance, for example, and an instance of `std::string` that has been used as the source of a move will still have a valid state, although no guarantees are made as to what that state is (in terms of how long the string is or what characters it contains).

A.1.2. Rvalue references and function templates

There's a final nuance when you use rvalue references for parameters to a function template: if the function parameter is an rvalue reference to a template parameter, automatic template argument type deduction deduces the type to be an lvalue reference if an lvalue is supplied or a plain unadorned type if an rvalue is supplied. That's a bit of a mouthful, so let's look at an example. Consider the following function:

```
template<typename T>
void foo(T&& t)
{}
```

If you call it with an rvalue as follows, then `T` is deduced to be the type of the value:

```
foo(42);          <- Calls foo<int>(42)
foo(3.14159);                                  Calls foo<double>(3.14159)
foo(std::string());     <- Calls foo<std::string>(std::string())
```

However, if you call `foo` with an *lvalue*, `T` is deduced to be an lvalue reference:

```
int i=42;
foo(i);                                  Calls foo<int&>(i)
```

Because the function parameter is declared `T&&`, this is therefore a reference to a reference, which is treated as just the original reference type. The signature of `foo<int&>()` is thus

```
void foo<int&>(int& t);
```